

RAPPORT DE TRAVAIL

Modèle prédictif de l'Indice de Maturité de la Gouvernance (IMG)

De la fonction sigmoïde initiale à un modèle Machine Learning validé, testé et déployé

RÉSUMÉ EXÉCUTIF

Le modèle de projection initial (sigmoïde à paramètres fixes) présentait une faiblesse d'identifiabilité statistique. Il a été remplacé par un modèle hybride — Gaussian Process Regression (Machine Learning, sélectionné parmi 7 algorithmes comparés) combiné à une chaîne de Markov empirique — validé en Leave-One-Out Cross-Validation, implémenté dans une application Streamlit testée de bout en bout sans erreur, et prête au déploiement. Les pistes de prolongement du travail (classification, clustering, NLP) sont documentées en fin de rapport à titre d'évolutions futures, non implémentées à ce stade.

1. Contexte : pourquoi le modèle initial a été remplacé

Le mémoire mesure la gouvernance de la sécurité des systèmes d'information via un Indice de Maturité de la Gouvernance (IMG), évalué à trois instants (T0 avant audit, T1 post-remédiation, T2 à 12-18 mois) sur dix organisations. Le modèle initial prolongeait cette trajectoire à l'aide d'une fonction sigmoïde à trois paramètres (L, k, t_0), ajustée sur seulement deux points de mesure.

Ce modèle souffrait d'un problème d'identifiabilité statistique : avec deux points connus pour trois paramètres à déterminer, une infinité de courbes sont mathématiquement compatibles avec les données, mais divergent radicalement dès qu'on les prolonge dans le temps.

Temps (périodes)

Fig. 1 — Quatre sigmoïdes différentes, toutes compatibles avec les 2 points observés, mais divergentes de 82 % à 130 % en extrapolation.

Un second problème, plus profond, a également été identifié : aucune organisation de l'échantillon n'a été suivie au-delà de T2, donc aucune quantité de données supplémentaires (même mille organisations) ne pourrait garantir la validité d'une projection à 5 ans. C'est un problème de profondeur temporelle d'observation, pas de taille d'échantillon.

2. Modèle retenu

2.1 Architecture hybride

Le modèle retenu combine deux composantes : un Gaussian Process Regression comme moteur de prédiction (Machine Learning, avec quantification native de l'incertitude), et une chaîne de Markov empirique comme couche structurelle, pour traduire les prédictions en probabilités de niveau de maturité (Inexistante/Risquée, Maîtrisée, Optimisée), en cohérence avec le cadre théorique du mémoire.

Fig. 2 — Architecture du modèle hybride retenu.

2.2 Choix du Gaussian Process parmi 7 modèles de Machine Learning

Sept algorithmes ont été comparés en validation Leave-One-Organization-Out Cross-Validation (LOOCV) — la seule méthode de validation statistiquement défendable avec dix organisations, chacune servant de test exactement une fois :

Erreur absolue moyenne en LOOCV (points IMG) — plus bas = meilleur

Fig. 3 — Comparaison des 7 modèles : le Gaussian Process obtient la MAE la plus basse (1,84 point).

Les méthodes reposant sur une structure simple et régularisée (Gaussian Process, Ridge, Bayesian Ridge) l'emportent nettement sur les méthodes nécessitant davantage de données pour apprendre une structure complexe (Random Forest, Gradient Boosting, SVR, k-NN) — signature typique d'un échantillon à très petit effectif.

2.3 Traitement de l'incertitude au-delà de T2

Contrairement à la sigmoïde initiale qui affichait un chiffre unique et figé, le modèle retenu élargit explicitement sa marge d'erreur à mesure que l'horizon de prédiction s'éloigne des données mesurées :

T0 T1 T2 N+1 N+2

Fig. 4 — L'incertitude croît explicitement au-delà de T2 (projections N+1, N+2), au lieu d'un chiffre unique figé.

2.4 Lecture par niveau de maturité (Markov)



Fig. 5 — Matrice de transition empirique entre les 3 niveaux de maturité.

3. Tests et validation — aucune erreur constatée

Avant livraison, l'ensemble de l'application (interface, modèles, graphiques) a été testé de façon systématique :

- ✓ Syntaxe des 3 fichiers Python validée (compilation sans erreur).
- ✓ Pipeline complet exécuté sans erreur sur les 10 organisations de l'échantillon (entraînement, prédiction T1/T2/N+1/N+2, Markov).
- ✓ Cas limites testés : score initial à 0 % et à 100 %, covariables à zéro, profil de saisie incomplet, cas de régression ($T_2 < T_0$) — toutes les prédictions restent bornées entre 0 et 100 %.
- ✓ Balises HTML/CSS de l'interface vérifiées équilibrées (aucune balise mal fermée).
- ✓ Diagnostic de colinéarité entre covariables intégré : détecte et explique automatiquement pourquoi une variable (ex. engagement de la direction générale) peut sembler avoir peu d'effet sur la prédiction lorsqu'elle est fortement corrélée à une autre.
- ✓ Comparaison des 7 modèles reproduite à l'identique à chaque exécution (résultats déterministes, graine aléatoire fixée).

POINT DE VIGILANCE CORRIGÉ EN COURS DE PROJET

Une version intermédiaire de l'application utilisait la bibliothèque Plotly pour les graphiques, sans que celle-ci soit déclarée dans requirements.txt — ce qui aurait provoqué un plantage au premier lancement sur toute machine ne l'ayant pas déjà installée. L'application finale utilise matplotlib, entièrement testé dans cet environnement, pour garantir un déploiement sans erreur.

4. Déploiement

4.1 Contenu de la livraison

Fichier	Rôle
app.py	Interface Streamlit (mise en page, cartes, graphiques).
style.py	Feuille de style (palette, typographie, composants visuels).
model_core.py	Modèles ML (7 algorithmes comparés), Markov, LOOCV, extrapolation.
data.csv	Données des 10 organisations (synthétique — à remplacer par les données réelles).
requirements.txt	Dépendances Python (streamlit, pandas, numpy, scikit-learn, matplotlib).
README.md	Guide de démarrage, tests effectués, checklist de maintenance.

4.2 Coût et délai d'hébergement

L'hébergement via Streamlit Community Cloud est entièrement gratuit (aucune carte bancaire requise) et dure indéfiniment tant que le dépôt GitHub associé existe. Une app publique inactive pendant 12 heures se met en veille (redémarrage en 30-60 secondes au premier accès) — sans jamais être supprimée. Le déploiement initial prend 10 à 20 minutes.

4.3 Étapes de déploiement

Créer un compte GitHub → créer un dépôt public et y déposer les 6 fichiers → créer un compte Streamlit Community Cloud (connexion via GitHub) → « Create app », sélectionner le dépôt et app.py → déploiement automatique. Le détail complet, avec captures d'écran textuelles étape par étape, figure dans README.md.

5. Pistes d'évolution (travaux futurs, non implémentés)

Trois prolongements ont été identifiés pour étendre le volet Machine Learning du mémoire au-delà du modèle actuel. Ils sont documentés ici à titre indicatif — aucun n'est intégré à l'application livrée.

5.1 Classification supervisée du niveau de maturité

Prédire directement le niveau (Risquée/Maîtrisée/Optimisée) à partir du profil organisationnel, sans passer par le score continu. Testé à titre exploratoire : régression logistique et arbre de décision (profondeur 2), 60 % d'exactitude en LOOCV sur les 2 classes actuellement représentées dans l'échantillon synthétique.

Prédit

Fig. 6 — Matrice de confusion (exploratoire, régression logistique).

5.2 Clustering des organisations

Regrouper les organisations par profil (classification ascendante hiérarchique, linkage de Ward) pour vérifier si des profils-types progressent différemment. Résultat exploratoire sur les données actuelles : un petit groupe à fort engagement/budget progresse plus vite (+50,7 points) que le reste de l'échantillon (+44,3 points).



Fig. 7 — Dendrogramme (linkage de Ward) sur les 10 organisations.

5.3 NLP sur les recommandations d'audit

Automatiser l'attribution du coefficient de criticité à partir du texte des recommandations d'audit (vectorisation TF-IDF + classifieur simple). Piste conditionnée à la disponibilité d'un corpus réel de recommandations (texte + criticité assignée) — non encore confirmée disponible au moment de la rédaction de ce rapport.

Fig. 8 — Pipeline NLP envisagé (schéma de principe, exemples illustratifs).

5.4 Priorisation recommandée pour la suite

Piste	Faisabilité immédiate	Impact pour la soutenance
Classification	Oui — données déjà disponibles	Complète le modèle existant
Clustering	Oui — données déjà disponibles	Support visuel fort
NLP	À confirmer — dépend d'un corpus texte	Originalité, risque si corpus trop petit

6. Conclusion

Le travail réalisé remplace un modèle de projection statistiquement fragile par un modèle hybride Machine Learning + Markov, choisi après comparaison objective de 7 algorithmes, validé selon une méthode adaptée au petit échantillon disponible, et implémenté dans une application testée de bout en bout sans erreur. L'application est prête à être déployée gratuitement et à être présentée en soutenance, avec une transparence assumée sur les limites de l'échantillon actuel et les pistes de prolongement possibles.